CSCI 6461 | Computer Systems Architecture | TOPIC 7

Machine Code & AArch64 Instruction Encoding

Instruction words, encoding fields, immediates, memory operations, and branch targets in AArch64

Dr. J. Burns

Topic 9 Outline

1. **Machine Code and 32-bit Instruction Words**
   - Assembly text, instruction words, bit fields, and byte order.
2. **AArch64 Encoding Families**
   - How high-order bits select data-processing, load/store, and branch formats.
3. **Data-Processing Encodings**
   - Register and immediate fields, worked encodings, and decoding an instruction word.
4. **Memory and Branch Encodings**
   - Scaled load/store offsets, PC-relative addresses, and immediate branch fields and targets.

Section 1

Machine Code and 32-bit Instruction Words

From Assembly to Machine Code

- A processor repeatedly **fetches**, **decodes**, and **executes** instructions.
- During fetch–decode–execute, the processor fetches a 32-bit instruction word from memory and interprets its bit fields.
- This topic starts with the earlier step: how an AArch64 assembly instruction is encoded as that fixed **32-bit** machine instruction.



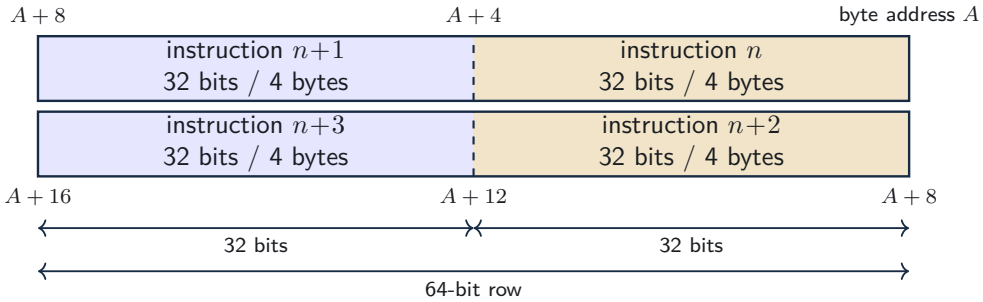
In memory, `0x8B020020` is stored as bytes `20 00 02 8B` in increasing address order (A64 instruction byte order).

AArch64 Encoding Essentials

- Every A64 instruction is a 32-bit word containing operation and operand fields.
- Different instruction families use different field layouts.
- Register fields are normally 5 bits, encoding register numbers 0–31.
- There are 31 general-purpose registers. Register number 31 denotes the zero register or stack pointer, depending on the instruction and operand.
- Identify the instruction form before interpreting its fields.

AArch64 Instruction Word Basics

Each AArch64 instruction is 32 bits, or 4 bytes. Four consecutive instructions occupy 16 bytes, shown here as two 8-byte rows.



Instruction Boundaries

Each instruction starts 4 bytes after the previous one. A 64-bit memory row holds two AArch64 instructions in this diagram; it is not a required hardware fetch width.

Section 2

AArch64 Encoding Families

How the Decoder Starts

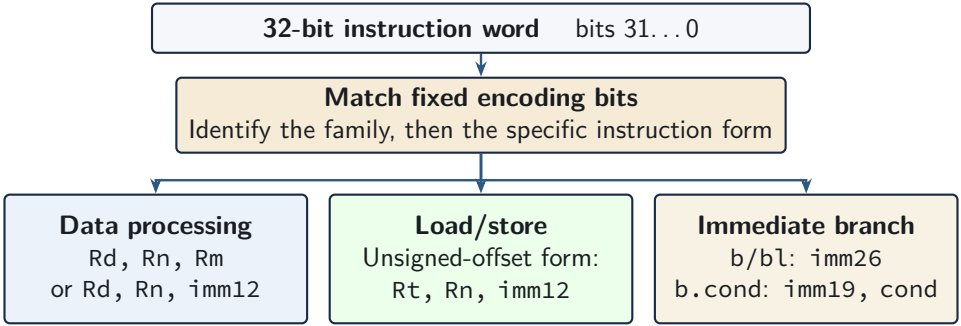
- AArch64 has several major encoding families.
- The decoder first checks high-order opcode bits to select the family.
- After the family is known, the remaining fields can be interpreted.

Examples of instruction families

- Data processing: arithmetic, logical, moves, shifts, and compares.
- Load/store: memory access through base registers and offsets.
- Branch: unconditional branches, calls, returns, and conditional branches.

Identify the broad instruction family first, then read the fields that belong to that family.

From Instruction Word to Operand Fields



The Form Determines the Meaning of the Fields

There is no universal opcode layout. The selected form determines how to interpret register numbers, immediates, shifts and offsets. The following slides expand these examples.

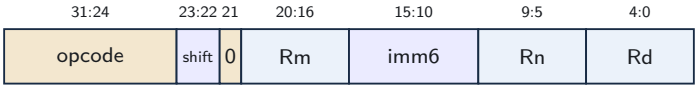
Section 3

Data-Processing Encodings

Data Processing: Register Form

Example Instruction

```
add x5, x6, x7
```



- `Rd` is the destination register.
- `Rn` is the first source register.
- `Rm` is the second source register.
- Bits 23:22 select the shift type; `imm6` (15:10) gives the shift amount. Bit 21 is fixed at 0.

Encoding Example: `add x5, x6, x7`

Register Numbers

$$Rd = x5 = 00101$$
$$Rn = x6 = 00110$$
$$Rm = x7 = 00111$$

Operation Fields

operation = add

operand size = 64-bit

shift amount = 0



The complete encoding is

`10001011000 00111 000000 00110 00101` = `0x8B0700C5`.

Data Processing: Immediate Form

Example Instruction

```
add x5, x6, #24
```



- The immediate field is 12 bits wide.
- The one-bit sh field (bit 22) selects a shift of 0 or 12 bits; bit 23 is fixed at 0.
- This format is common for address arithmetic and small integer constants.

Encoding Example: `add x5, x6, #24`

Operands

$$Rd = x5 = 00101$$
$$Rn = x6 = 00110$$
$$\text{imm12} = 24 = 000000011000$$

Instruction Meaning

```
x5 = x6 + 24
```

The immediate value is part of the instruction word.



`add x5, x6, #24` is encoded as `0x910060C5`.

AArch64 Immediate Encodings Are Specialized

- Different instruction families have different immediate encodings.
- Some constants fit directly in an add or sub immediate.
- Other constants require move-wide instructions such as `movz`, `movn`, or `movk`.
- Logical instructions use their own bitmask-immediate encoding.

Immediate Fields

Do not ask whether a constant fits "in AArch64" in general. Ask whether it fits in the specific immediate field of the instruction being used.

Decoding Checklist

1. Convert the hexadecimal word to 32 binary bits.
2. Identify the broad encoding family from the opcode bits.
3. Split the word into the fields for that family.
4. Translate register fields into x or w register names.
5. Interpret immediates carefully: sign extension, scaling, and shifts matter.
6. Reconstruct the assembly instruction.

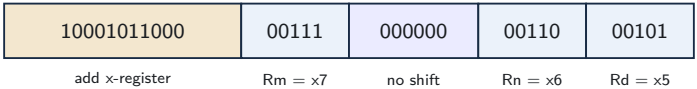
Important

The same bit positions can mean different things in different instruction families. Always identify the family before naming the fields.

Reverse Engineering Example

Given Machine Code

`0x8B0700C5`



Decoded Instruction

`0x8B0700C5` $\longrightarrow$ `add x5, x6, x7`

Section 4

Memory and Branch Encodings

Load/Store Instruction Encoding

Example Instruction

```
ldr x5, [x6, #16]
```



- `Rt` is the register loaded or stored.
- `Rn` is the base address register.
- `imm12` is a scaled unsigned offset for this common form.
- For a 64-bit `ldr`: address $=$ x6 $+ 8 \times$ imm12.

Encoding Example: `ldr x5, [x6, #16]`

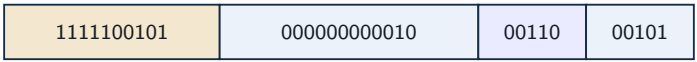
Operands

$$Rt = x5 = 00101$$
$$Rn = x6 = 00110$$
$$\#16 = 2 \times 8 \text{ bytes}$$

Encoded Offset

$$\text{imm12} = 2 = 000000000010$$

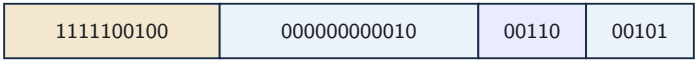
The instruction stores the scaled value, not the byte offset directly.



`ldr x5, [x6, #16]` is encoded as `0xF94008C5`.

Store Uses the Same Address Fields

`str x5, [x6, #16]` is encoded as `0xF90008C5`.



- The base register and scaled offset fields match the load example.
- The operation field changes from load to store.
- `Rt` is now the source register whose value is written to memory.

Scaled Offsets Depend on Access Size

Unsigned Offset Form

$$\text{address} = \text{base} + \text{imm12} \times \text{access size}$$

Instruction kind	Access size	Encoded immediate meaning
`ldrb/strb`	1 byte	byte offset
`ldrh/strh`	2 bytes	byte offset divided by 2
`ldr w/str w`	4 bytes	byte offset divided by 4
`ldr x/str x`	8 bytes	byte offset divided by 8

Scaled Offset Range

Scaling lets a 12-bit field cover a larger byte range for naturally aligned accesses.

PC-Relative Addresses in AArch64

- PC-relative instructions use the current instruction address as their reference; `adrp` uses its 4 KiB page base.
- This supports position-independent code and nearby constants or labels.
- For `adr`, literal loads and immediate branches, PC is the address of the current instruction.

Common PC-Relative Forms

- `adr`: forms an address near the current PC.
- `adrp`: forms a page address relative to the current PC page.
- `ldr literal`: loads data from a nearby PC-relative literal location.
- `b` and `bl`: branch to a PC-relative target.

Literal Loads in AArch64

Concept

A literal load reads a constant stored near the instruction stream using a PC-relative offset.

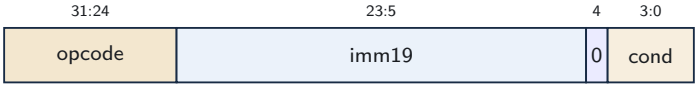


- The instruction encodes where to find the constant relative to the PC.
- The loaded value is data, even though it may be placed near the code.
- The pseudo-instruction `ldr x0, =constant` may use a literal pool; `ldr x0, literal` explicitly loads from a label.

Conditional Branch Encoding

Example Forms

```
b.eq label    b.lt label    b.ne label
```



- The condition code is encoded in the low 4 bits.
- The signed immediate is 19 bits, scaled by 4 bytes.
- Conditional branches test the flags set by earlier compare or arithmetic instructions.

Conditional Branch Example: `b.eq`

Target Offset

current PC = `0x100`

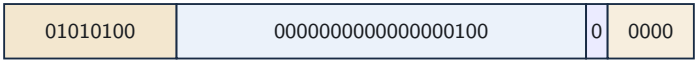
target = `0x110`

imm19 $= 4$

Condition Field

eq $= 0000$

The branch is taken when the Z flag is set.



`b.eq target` is encoded as `0x54000080`.

Unconditional Branch Encoding

Instruction Forms

```
b label        bl function
```



- `b` and `bl` use a signed 26-bit immediate field.
- The offset is measured in 4-byte instructions, not raw bytes.
- `bl` also writes the return address (PC + 4) into `x30`.

Target Calculation

$$\text{target address} = \text{PC} + \text{signextend}(\text{imm26}) \times 4$$

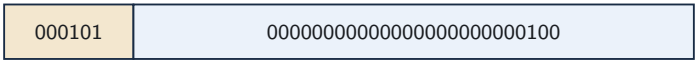
Branch Example: Forward Target

Code Shape

```
0x100:  b target
0x104:  add x0, x0, #1
0x108:  sub x1, x1, #1
0x10c:  nop
0x110:  target:
```

Offset

$$\Delta = \texttt{0x110} - \texttt{0x100}$$
$$= 16 \text{ bytes}$$
$$\text{imm26} = 16/4 = 4$$



`b target` is encoded as `0x14000004`.

Common Encoding Mistakes

- Treating a register number as decimal text instead of a binary field.
- Forgetting that AArch64 register fields are usually 5 bits.
- Encoding a byte offset directly when the immediate field is scaled.
- Adding a pipeline offset to PC instead of using the branch instruction address.
- Reading little-endian memory bytes as if they were the printed instruction word.

Practical Habit

Write the bit ranges first, then fill in the fields. Do not try to guess the final hexadecimal word too early.

Topic 9: Summary

- AArch64 instructions are fixed-width 32-bit words.
- The meaning of each bit field depends on the instruction family.
- Data-processing register instructions encode register operands and optional shift information.
- Data-processing immediate instructions use a different field layout.
- Load/store instructions commonly encode a base register, target/source register, and scaled offset.
- Direct immediate branches encode PC-relative offsets scaled by 4 bytes; register-indirect branches use a register target.
- In `b.cond`, bits 3:0 encode the condition and `imm19` encodes the signed instruction displacement.

Self-Test Questions: Instruction Words

1. How many bytes does an A64 instruction occupy? How far apart are consecutive instruction addresses?
2. If the instruction word is `0x8B020020`, what are its four bytes in increasing memory-address order?
3. Why must the instruction family be identified before interpreting operand fields?
4. What do `Rd`, `Rn`, and `Rm` represent in `add x5, x6, x7`? Give their 5-bit register numbers.
5. How do the fields change for `add x5, x6, #24`? What does its `sh` bit control?
6. Decode `0x8B0700C5` into an assembly instruction using the register-form layout.

Self-Test Questions: Memory and Branches

1. In `ldr x5, [x6, #16]`, why is `imm12` equal to 2? Which field identifies the base register?
2. How does the role of `Rt` differ between `ldr` and `str`?
3. For a 32-bit unsigned-offset load with `imm12 = 3`, what is the byte offset?
4. What address is the reference for a literal load, and what page is the reference for `adrp`?
5. A branch at `0x100` targets `0x110`. What is its encoded immediate? Is the reference PC or PC + 4?
6. Compare `b` and `b.eq`: give the immediate-field width, offset scaling, and location of the condition field where applicable.

Looking Ahead

Next Topic

Building a simple single-cycle RISC datapath.

- Instruction fields become inputs to the datapath and control logic.
- Register fields select operands in the register file.
- Immediate fields must be extracted, extended, and routed.
- Branch offsets feed the next-PC calculation.